

Проверка в Editor (не на Build)

Стресс-тест - около 100 целей, большая частота выстрелов

Вариант 1

targets speed 5, period 0.2 (rate 5)

projectile speed 20, lifetime 4

Targets Amount - 200

ShootRate - 50

~100 targets, ~320 projectiles

Mode	Naive	Pool
FPS	38-42	38-44
Frame	~26ms	~24ms
Max instantiate calls	6.11 ms (6 calls)	0
Max destroy calls	2.58 ms (14 calls)	0

Вариант 2 (более быстрые объекты)

targets speed 10, period 0.1 (rate 10)

projectile speed 40, lifetime 1

Targets Amount - 200

ShootRate - 100

~100 targets, ~100 projectiles

Mode	Naive	Pool	Pool (before prewarm)
FPS	39	40-44	41
Frame	~25ms	~23ms	23
Max instantiate calls	6.11 ms (6 calls)	0	5.38ms (6 calls), stops at 92 frame
Max destroy calls	2.58 ms (14 calls)	0	0

Пояснения к компонентам для выставки теста:

Режим теста запускается при выборе `IsTargetsTestOn` и/или `IsProjectilesTestOn` на компоненте `TestSpawnAdjuster` объекта `PerformanceTest`

Объект `PerformanceTest`:

1. компонент `SpawnModeController` - выбор mode

2. компонент `TestSpawnAdjuster`

- `Targets`

параметры для `SpawnPoints` ("точка" спауна цели), которые при тесте будут выставляться в необходимом количестве в зоне спауна целей (существующие по умолчанию при тесте деактивируются). Частота выхода целей из `SpawnPoint` и скорость целей - в структуре `TestSpawnData`

- `Projectiles`

Скорость, время жизни снарядов в режиме теста

- `TestParameters`

`IsTargetsTestOn`, `IsProjectilesTestOn` - если отключены, то соответственно спаун целей / снарядов по умолчанию, то есть, как задано в параметрах объектов внутри `Gameplay` объекта

`TargetsAmount` (при `IsTargetsTestOn = true`) - сколько целей должно быть на сцене при тесте (так как часть целей уничтожается, нужно подбирать больше, чем желаемое число)

`ShootRate` (при `IsProjectilesTestOn = true`) - частота выстрелов (исходя из неё, скорости снарядов и расстояния от турели до зоны целей по оси `z`) рассчитывается размер `prewarm` для пула снарядов

Часть 2

Что сделано

1. Сделаны MovableSystem и LifetimedSystem и интерфейсы IMovable, ILifetimed для контроля в этих системах

В Target и Projectile теперь сами не вызывают Update(), имплементируют интерфейсы и контролируются системами. На остальных игровых объектах уже были корутины

2. В FPSCounter (переименован из PerformanceCounter) - замена проверок каждый кадр на проверку в корутине плюс удалён отдельный таймер для расчёта времени кадра

В остальных системах, как на сцене, так и в UI, уже реализовано через корутины или events

3. TargetsService и ProjectilesService сделаны обычными C#-классами, так же из них убрана информация с именами заголовков, нужными для отображения в UI

4. Разделены Canvas для динамических и статических объектов

Разница

По времени:

Медианное время UpdateScene снизилось на 0.79 ms

Время на BehaviourUpdate:

В предыдущей версии - 0.35..1.7 ms

В нынешней версии - 0.44..0.78 ms

То есть, время более стабильное и пиковое значение намного меньше

Общая частота кадров стала выше

По архитектуре:

Пришлось незначительно изменить код классов, которые до этого сами вызывали Update(). С другой стороны, появилось обобщение для объектов, которые движутся, у которых есть время жизни, что может в будущем упростить само продумывание новых классов с подобным функционалом.

Увеличилось количество полей в GameInitializer, для инициализации в бывшие MonoBehaviour

Архитектура и понимание кода скорее немного усложнились